

캠퍼스런치

코드베이스 디버깅 가이드

작성일: 2026-07-23
목적: 파일 구조, 모듈 연결, DB 연결, 코딩·디버깅 방법, 기능별 플로우
대상: develop 브랜치 (파트너 pull + 버그 수정 + 사장님 탭 개편 반영)

목차

1. 프로젝트 한눈에 보기
2. 레이어 구조 (화면 -> 상태 -> 데이터 -> DB)
3. lib/ 폴더별 역할
4. 앱 시작·화면 전환 흐름
5. 기능별 플로우
 - 5.1 회원·인증
 - 5.2 혼잡도 (표시·제보·계산·실시간)
 - 5.3 사장님 (코드 인증·탭·입장 인원)
 - 5.4 관리자
 - 5.5 푸시·북마크·홈 필터
 - 5.6 리워드(스탬프·기프트콘)
 - 5.7 APK 테스트·소셜 로그인
6. Supabase SQL 맵
7. DB 연결 상세 (앱 <-> Supabase)
8. 코딩 컨벤션
9. 디버깅 방법
10. 외부 서비스·환경 변수
11. 증상별 디버깅 체크리스트
12. 최근 주요 변경 요약

1. 프로젝트 한눈에 보기

Flutter 앱 + Supabase 백엔드 구조입니다.

상태 관리는 Provider 패턴 하나(AppProvider)로 중앙 집중됩니다.

화면(Screens)은 AppProvider를 watch/read 하고, DB 접근은 data/ 레포지토리가 담당합니다.

```
campuslunch-app/  
lib/      ← Flutter 앱 (UI + 상태 + 데이터)  
supabase/ ← SQL (테이블·RPC·RLS·트리거)  
docs/     ← 설정·가이드 문서  
tool/     ← 시드·유틸 스크립트  
test/     ← 단위 테스트  
.env      ← Supabase URL, API 키 (Git 포함)  
.env.secrets ← Secret 키 (시드용, Git 포함 private repo)
```

2. 레이아웃 구조

데이터는 항상 아래 방향으로, UI 이벤트는 위에서 아래로 흐릅니다.

```
[ Screens / Widgets ] home_screen, detail_screen, owner_screen ...  
    | onTap / watch  
    v  
[ AppProvider ]      stage, restaurants, reportStatus, verifyOwnerCode ...  
    | repository 호출  
    v  
[ data/ + services/ ] supabase_restaurant_repository, profile_repository ...  
    | SupabaseClient RPC / from('table')  
    v  
[ Supabase ]        PostgreSQL + Auth + Realtime + Storage
```

혼잡도 계산은 DB 트리거(recalculate)가 주력이고, 앱은 crowd_status_algorithm.dart로 클라이언트 폴백·표시를 합니다.

3. lib/ 폴더별 역할

main.dart

앱 진입점. .env 로드 -> Kakao/Google/Supabase/Push 초기화 -> AppProvider.init()

providers/app_provider.dart

* 핵심. stage(화면 단계), 로그인, restaurants 목록, 제보, 사장님, 북마크, 푸시, Realtime 구독, admin 설정 전부 여기.

screens/

화면 단위 UI. main_screen=하단탭, home/map/my/owner/admin/login 등.

widgets/

재사용 UI. restaurant_card, report_sheet, owner_verify_sheet, status_badge 등.

data/

DB 접근·매핑. supabase_restaurant_repository(매장·혼잡도), profile_repository(users), auth_repository, analytics_repository, reward_repository.

data/crowd_*

혼잡도 전용. level_mapper(enum<->한글), algorithm(v2 계산), helpers(제보 집계).

services/

외부 SDK 래퍼. supabase_service, kakao/google_auth, push_notification, places.

models/

데이터 클래스. Restaurant, Account, CrowdReport, OwnerSeatUpdate, Reward 등.

utils/

순수 헬퍼. business_hours, available_restaurant_ranking, report_feedback, 라벨.

config/env.dart

SUPABASE_URL, ANON_KEY, Kakao, Google Maps/OAuth 키 읽기.

4. 앱 시작·화면 전환 흐름

```
main() -> AppProvider.init()  
  +- Supabase 세션 있음 -> _onSupabaseSignedIn -> stage=app/admin  
  +- 로컬 세션 있음 -> prefs 복원 -> stage=app/admin  
  +- 없음 -> onboarding 또는 login
```

AppProvider.stage -> main.dart _Root switch:

```
splash | onboarding | login  
location_permission | notification_permission  
app -> MainScreen (하단 탭)  
admin -> AdminScreen
```

MainScreen 하단 탭:

• 일반 유저: [홈] [지도] [MY]

- 사장님(hasOwnerTab): [사장님] [홈] [지도] [MY] — 인증 후 기본 탭=사장님
IndexedStack으로 탭 전환 (상태 유지).

5. 기능별 플로우

5.1 회원·인증

LoginScreen

- > AppProvider.login / register / loginWithKakao / loginWithGoogle
- > Supabase Auth + ProfileRepository.upsertFromAuthUser (public.users)
- > _saveSession (SharedPreferences + stage 결정)

RPC: email_signup_status, oauth_login_email_check, is_nickname_available

탈퇴: delete_own_account

5.2 혼잡도

[표시]

SupabaseRestaurantRepository.fetchAll()

- ← restaurants + crowd_reports + crowd_status
- > _mergeRow() + computeStatusFromReports()
- > Restaurant.status (여유로움/약간혼잡/자리없음/영업안함)
- > HomeScreen / MapScreen / DetailScreen / RestaurantCard

[제보]

ReportSheet -> report_feedback.submitCrowdReportFeedback()

- > AppProvider.reportStatus()
 - source = owner (본인 매장+role owner) | user (그 외)
 - user: 5분 콜다운 + GPS 150m (AppProvider, kDebugMode 우회)
 - 제한 검증: 서버 RPC X, 클라이언트 전달
- > RPC submit_crowd_report
- > crowd_reports INSERT -> 트리거 -> recalculate_crowd_status

[실시간]

AppProvider._subscribeRealtime()

crowd_status | crowd_reports | owner_seat_updates 변경 -> fetchAll()

DB enum: crowd_level (normal/full/closed), crowd_source (user/owner/system). 한글 UI는 metadata.status + CrowdLevelMapper로 변환.

5.3 사장님

[코드 인증]

MyScreen / OwnerVerifySheet -> verifyOwnerCode()

- > RPC claim_owner_by_code (로그인 필수)
- > restaurants.owner_id, users.role=owner
- > stage=app, mainTabIndex=0 (사장님 탭)

[혼잡도 변경]

OwnerScreen -> reportStatus() — 본인 매장만 owner source

[입장 인원]

OwnerScreen -> submitOwnerSeatUpdate()

- > RPC submit_owner_seat_update
- > DetailScreen: get_owner_seat_update (1시간 표시)

소유 매장 ID: fetchOwnedRestaurantIds() (restaurants.owner_id) + JWT metadata restaurant_ids 폴백.

5.4 관리자

login admin/admin123 (debug) 또는 Supabase role=admin

- > AdminScreen
- > fetchMetrics (admin_dashboard_metrics)
- > 매장 CRUD, owner_code 생성, manual_rank, owner_influence
- > AdminMapRegisterTab + PlacesService (Google Places)

5.5 푸시·북마크·홈

[푸시] FCM 원격 푸시 (앱 종료 시에도 수신)

- FcmPushService — 토큰 upsert + 포그라운드 로컬 표시
- Edge: send-peak-push (cron), send-community-push (덧글)
- 테이블: user_push_tokens, user_notification_prefs, peak_push_sent_log
- SQL: fcm_push.sql / 배포: docs/FCM_PUSH_SETUP.md
- 어드민 PushSettingsPage -> push_notification_config

[북마크] toggleBookmark -> SharedPreferences + Auth metadata

BookmarkListScreen, HomeScreen 북마크 필터

[홈 필터] HomeScreen 로컬 state

- 정렬·지역·카테고리·검색·북마크만
- available_restaurant_ranking.dart

5.6 리워드

제보 성공 -> submit_crowd_report (rewards.sql) -> 스탬프 JSON

- > AppProvider.lastStampResult, UserReward 로컬 갱신

```
-> RewardScreen (MY 탭), redeem_gifticon RPC  
lib/data/reward_repository.dart, lib/models/reward.dart
```

스탬프 일일 한도: grant_stamp 하루 최대 3 (KST). 출시 SQL: rewards_daily_cap_to_3.sql

5.7 APK 테스트·소셜 로그인

```
flutter build apk --release  
key.properties 있음 -> 릴리스 keystore 서명  
없음 -> debug keystore로 release APK 서명
```

```
flutter run(디버깅) OK, APK만 실패 -> 코드 버그가 아니라  
OAuth 콘솔에 APK 서명 SHA-1/키 해시 미등록 가능성 큼
```

```
Google: Cloud Console Android OAuth + SHA-1  
cd android && ./gradlew :app:signingReport  
Kakao: developers.kakao.com Android 키 해시  
dart run tool/print_kakao_android_key_hash.dart  
dart run tool/print_kakao_android_key_hash.dart --release
```

「로그인이 취소되었습니다」 = SDK가 설정 오류를 cancel로 반환하는 경우 많음

6. Supabase SQL 실행 순서

1. users_auth.sql users + Auth 트리거
2. policies.sql RLS (restaurants, crowd_reports)
3. rpc_email_signup_status.sql
4. rpc_claim_owner.sql 사장 코드 인증
5. rpc_delete_own_account.sql
6. rpc_nickname_available.sql
7. rpc_oauth_login_email_check.sql
8. analytics_events.sql
9. crowd_status.sql 헬퍼 + submit_crowd_report
10. crowd_status_v2_compute.sql v2 계산 + 트리거
11. push_analytics.sql
12. fcm_push.sql (토큰·선택·피크/커뮤니티 FCM RPC)
12. owner_seat_updates.sql 입장 인원
13. rewards.sql 스탬프·기프트콘 (submit_crowd_report 재정의)

crowd_status.sql 하단: Realtime publication 추가 블록 (재실행)

hotfix_crowd_report_types.sql — enum 타입 오류 시 one-shot

schema.sql — 문서용 (실행 X)

주요 테이블: users, restaurants, crowd_reports, crowd_status, owner_seat_updates, analytics_events, gifticons, system_settings.

7. DB 연결 상세 (앱 <-> Supabase)

앱은 supabase_flutter SDK 하나로 PostgreSQL·Auth·Realtime·Storage에 접속합니다. 화면은 DB를 직접 호출하지 않고, 항상

Repository 경유입니다.

[연결 초기화]

```
main.dart
  dotenv.load('.env')
  Env.isSupabaseConfigured (URL + ANON_KEY 둘 다 있어야 true)
  SupabaseService.initialize()
    -> Supabase.initialize(url, anonKey)
    -> SupabaseService.client == Supabase.instance.client
```

[요청 경로]

```
Screen/Widget
-> AppProvider (비즈니스 로직·세션·캐시)
-> *Repository (SupabaseClient 주입 가능, 기본=SupabaseService.client)
-> client.from('table') | client.rpc('fn') | client.storage
-> PostgREST + RLS + PostgreSQL 함수
```

7.1 Auth 연동

Supabase Auth (auth.users)

email OTP / Kakao / Google -> JWT 세션
trigger handle_new_user() -> public.users 자동 upsert

앱 로그인 후:

```
ProfileRepository.upsertFromAuthUser() (닉네임·provider 동기화)
ProfileRepository.fetch() (role, nickname)
AppProvider._saveSession() (SharedPreferences 백업)
```

JWT role: app_metadata.role / user_metadata.role

admin 판별: SupabaseService.isAdmin
owner 매장: restaurants.owner_id = auth.uid()

7.2 테이블·RPC 매핑

Repository DB 접근

```
SupabaseRestaurantRepo restaurants (SELECT/CRUD)
  crowd_reports (SELECT/INSERT/RPC)
  crowd_status (SELECT)
  owner_seat_updates (SELECT/RPC)
  system_settings (SELECT/UPSERT)
  storage (매장 이미지 업로드)
  RPC: submit_crowd_report, claim_owner_by_code,
  submit_owner_seat_update, admin_dashboard_metrics
ProfileRepository public.users (SELECT/UPDATE/UPSERT)
  RPC: is_nickname_available, delete_own_account
AuthRepository RPC: email_signup_status,
  oauth_login_email_check
AnalyticsRepository analytics_events (INSERT)
RewardRepository gifticons, RPC: redeem_gifticon
Push (간접) RPC: record_push_event
```

7.3 RLS·보안

anon key는 클라이언트에 노출되므로, 접근 제어는 RLS + SECURITY DEFINER RPC로 합니다.

- users: 본인 row만 SELECT/UPDATE (auth.uid() = id)
- crowd_reports: INSERT는 RPC submit_crowd_report 경유 (소유·GPS·쿨다운 검증)
- restaurants: owner_id 변경은 claim_owner_by_code RPC만
- admin CRUD: is_admin() 함수 + JWT role=admin

Secret key(.env.secrets)는 앱에 넣지 않음 — tool/ 시드·관리 스크립트 전용.

7.4 Realtime·로컬 폴백

```
[Realtime] AppProvider._subscribeRealtime()
  channel 'crowd_realtime'
  postgres_changes: crowd_status (all)
  crowd_reports (insert)
  owner_seat_updates (insert)
-> _loadRestaurantsFromSupabase() -> notifyListeners()
```

[Supabase 미설정/로드 실패]

initialRestaurants (lib/data/restaurants.dart) 시드 데이터
SharedPreferences cl_restaurant_overrides (혼잡도 로컬 저장)
제보: RPC 대신 로컬 override만 갱신

_supabaseRestaurantsLoaded == true 이면 override 무시

7.5 Dart 모델 <-> DB 컬럼

Restaurant (Dart) restaurants + crowd_status + reports 집계
id, name, category, area, status(한글), updated(분)

CrowdReport crowd_reports
level: crowd_level enum (normal/full/closed/relaxed)
source: crowd_source enum (user/owner/system)
metadata.status: 한글 UI ('여유로움' 등) — CrowdLevelMapper

UserProfile public.users
role: user | owner | admin
provider: email | kakao | google

8. 코딩 컨벤션

이 프로젝트는 '단일 Provider + Repository' 패턴을 일관되게 따릅니다. 새 기능 추가 시 아래 규칙을 지키면 디버깅·리뷰가 쉬워집니다.

상태 관리

전역 상태는 AppProvider(ChangeNotifier) 하나. 화면은 context.watch(리빌드) / context.read(이벤트)만 사용. 홈 필터처럼 화면 전용

state는 StatefulWidget 로컬 state 허용.

레이어 분리

Screens/Widgets: UI + 사용자 입력만. AppProvider: 검증·세션·목록 갱신·Realtime. data/*Repository: Supabase 호출·JSON 매핑. utils/: 순수 함수(영업시간, 정렬, 라벨). Screen에서 SupabaseClient 직접 import 금지.

Repository 패턴

각 Repository는 SupabaseClient? client 생성자 주입 (테스트 mock용). 기본값 SupabaseService.client. try/catch + debugPrint('태그 ...') 후 null/기본값 반환 또는 rethrow.

에러·피드백

AppProvider 메서드는 String? err 반환 (null=성공). UI는 report_feedback.dart처럼SnackBar 표시. async 후 context.mounted 체크 필수 (admin_screen 등).

로컬 저장

SharedPreferences: 세션, 북마크, 푸시 설정, owner IDs. Supabase 로드 성공 시 cl_restaurant_overrides는 쓰지 않음. 키 이름은 AppProvider 상단 _k* 상수로 관리.

혼잡도 이중 계산

DB: crowd_status_v2_compute.sql 트리거가 정본. 앱: crowd_status_algorithm.dart + helpers는 fetchAll 병합·폴백. SQL과 Dart 알고리즘 불일치 가능 — 버그 시 양쪽 비교.

디버그 전용

kDebugMode: admin/admin123, owner/owner123 로컬 계정. 제보 GPS·쿨다운 자동 우회(flutter run 테스트). debugPrint 태그: [Supabase] [Profile] [Auth] [Kakao] [Google] [Analytics].

파일 배치

screens/ = 전체 화면, widgets/ = 재사용 조각, models/ = 불변 데이터 클래스(copyWith), constants/ = 이메일 OTP 등 고정값, config/ = Env.

8.1 새 기능 추가 순서 (권장)

1. supabase/*.sql — 테이블/RPC/RLS 정의 (Dashboard에서 실행)
2. lib/models/ — Dart 모델
3. lib/data/*_repository.dart — DB 호출
4. lib/providers/app_provider.dart — 상태·비즈니스 로직
5. lib/screens/ 또는 widgets/ — UI
6. test/ — 알고리즘·순수 로직 단위 테스트 (선택)

9. 디버깅 방법

증상이 나왔을 때 '어디를 먼저 볼지' 순서입니다. 아래 11장 체크리스트와 함께 쓰면 됩니다.

9.1 Flutter 앱 디버깅

실행: flutter run (또는 IDE Run)
로그: Debug Console에서 태그 필터
[Supabase] Auth·세션·매장 로드
[Profile] users 테이블
[Auth] RPC email/oauth

Release APK: adb logcat | grep -E 'Kakao|Google|SignIn'
Hot reload: UI 변경. Hot restart: init()·세션 재실행.
Provider 상태 확인: AppProvider 필드 breakpoint
restaurants, _userRole, _ownerRestaurantIds, stage

9.2 Supabase Dashboard

Table Editor
restaurants.owner_id, users.role 확인
crowd_reports.level/source/metadata
crowd_status 최신 row

SQL Editor
supabase/*.sql 순서대로 재실행
RPC 수동 호출: select submit_crowd_report(...)

Authentication > Users
email confirmed?, provider, metadata.role

Database > Replication
crowd_status, crowd_reports Realtime 활성화 확인

Logs (API / Postgres)
RLS 거부, enum 타입 오류, RPC not found

9.3 단위 테스트

flutter test
test/crowd_status_algorithm_test.dart
-> 혼잡도 v2 알고리즘 (DB 트리거와 대조)
test/widget_test.dart

알고리즘 변경 시: Dart 테스트 + SQL recalculate 결과 비교

9.4 단계별 추적 (기능별)

[혼잡도 표시 안 맞음]

- 1) Dashboard crowd_status vs 앱 Restaurant.status
- 2) fetchAll() _mergeRow + computeStatusFromReports
- 3) Realtime 구독 동작? stale fetch (_refreshGen)?
- 4) Supabase 로드했는데 override prefs 잔존?

[제보 실패]

- 1) 콘솔 AuthException / RPC error message
- 2) submit_crowd_report 배포 + enum 타입
- 3) user: GPS 150m + 5분 쿨다운 (AppProvider, kDebugMode 우회)
- 4) owner: _ownsRestaurant + role=owner

[사장님 인증]

- 1) 로그인 상태? claim_owner_by_code -> LOGIN_REQUIRED
- 2) restaurants.owner_code, owner_id
- 3) users.role = owner
- 4) fetchOwnedRestaurantIds() 결과 vs _ownerRestaurantIds

9.5 알려진 함정

- report_source enum 없음 — 실제 타입은 crowd_source
- profiles 테이블 없음 — public.users 사용
- 영업시간: 앱은 description JSON hours_periods, SQL은 hours 문자열만 — 영업안함 판정 불일치 가능
- rewards.sql 실행 시 submit_crowd_report 함수 재정의 — 순서 주의
- 스탬프 일일 한도 3 — supabase/rewards_daily_cap_to_3.sql 적용
- APK 소셜 로그인 실패 -> signingReport / 카카오 릴리스 키 해시 등록
- Supabase 미설정 시 Realtime·RPC 전부 스킵, 로컬만 동작

10. 외부 서비스·환경 변수

.env: SUPABASE_URL, SUPABASE_ANON_KEY, KAKAO_NATIVE_APP_KEY, GOOGLE_MAPS_API_KEY, GOOGLE_OAUTH_*

.env.secrets: SUPABASE_SECRET_KEY (시드·관리 스크립트용)

android/key.properties + keystore: 릴리스 APK 서명 (git 제외)
Supabase 미설정 시 -> initialRestaurants 시드 데이터 + 로컬 SharedPreferences만 사용.

11. 증상별 디버깅 체크리스트

- 혼잡도 제보 실패 -> submit_crowd_report RPC 배포? crowd_level/crowd_source enum?
- 사장님 '본인 매장만' -> owner_id 연결? 로그인 후 코드 재인증?
- 표시가 이상함 -> Realtime 갱신 vs 로컬 override(cl_restaurant_overrides) 잔존?
- 영업안함 vs 여유로움 -> business_hours / hours_periods 불일치 (앱 vs SQL)?
- 사장 홈 제보 -> 본인 매장=owner source, 타 매장=user source (GPS 필요)
- Auth 문제 -> public.users row 존재? email 미인증?
- Admin CRUD 실패 -> is_admin() / JWT role / RLS
- APK 카카오/구글 -> 로그인 취소? -> 릴리스 SHA-1 키 해시 콘솔 등록

12. 최근 주요 변경 요약

[75aa50a] 제보 제한 AppProvider 전달, kDebugMode 우회, GPS 150m
어드민 제보 제한 Supabase 토글 제거, 24h 영업 business_hours 수정
Realtime publication SQL, 스텝프 일일 한도 3 (rewards_daily_cap_to_3.sql)
[48dd69d] Android 릴리스 서명 key.properties
[4d4bdb3] 리워드 시스템 (RewardScreen, rewards.sql)
[이전] 홈 필터/정렬, Realtime 구독, 사장님 mainScreen 탭, owner enum 수정

핵심 파일 빠른 참조

상태 허브	lib/providers/app_provider.dart
매장-혼잡도	lib/data/supabase_restaurant_repository.dart
제보 UI	lib/widgets/report_sheet.dart
제보 연결	lib/utills/report_feedback.dart
혼잡도 계산	lib/data/crowd_status_algorithm.dart supabase/crowd_status_v2_compute.sql
사장 RPC	supabase/rpc_claim_owner.sql supabase/owner_seat_updates.sql
리워드	lib/data/reward_repository.dart, lib/screens/reward_screen.dart supabase/rewards.sql
하단 탭	lib/screens/main_screen.dart